

BSG INSTITUTE

Certified AI/LLM Solution Architect

Módulo 1 — Fundamentos de Arquitecturas LLM

Manual de Instalación

Gateway LLM Propio con Seguridad de Nivel Producción

Gateway LLM Propio con Seguridad de Nivel Producción · Linux y GitHub

Elaborado por:

Julio Cesar Murrugarra Velasquez

Lima, Perú — Octubre 2026

Proyecto Final - Opción 4 de 10

Repositorio: [**github.com/cesarmv85-del/fundamentos-arquitectura-LLm-gateway-owasp**](https://github.com/cesarmv85-del/fundamentos-arquitectura-LLm-gateway-owasp)

Contenido

1. Entornos de instalación	4
2. Qué se instala.....	4
3. Prerrequisitos.....	5
3.1 Software.....	5
3.2 Recursos de la máquina	5
3.3 Red	6
3.4 Versiones verificadas	6
4. Opción A — GitHub Codespaces.....	6
Paso 1 · Crear el codespace	6
Paso 2 · Verificar la instalación	7
Paso 3 · Arrancar el gateway	7
Paso 4 · Probar desde una segunda terminal	7
Paso 5 · Acceder desde el navegador o desde fuera.....	7
Paso 6 · Claves de proveedores como secretos (opcional)	8
Paso 7 · Detener y eliminar	8
5. Opción B — Máquina Linux propia.....	8
Paso 1 · Instalar los prerrequisitos	8
Paso 2 · Descargar el código	8
Paso 3 · Ejecutar el instalador.....	9
Paso 4 · Verificar	9
Paso 5 · Arrancar el gateway	10
Instalación manual (alternativa al instalador).....	10
6. Opción C — GitHub Actions	10
7. Verificación de la instalación	11
8. Configuración	12
8.1 Proveedores.....	12
8.2 Clientes y límites de consumo	13
8.3 Registros, navegador y entorno	13
8.4 Interruptores de demostración	14
9. Conectar un proveedor real.....	14
10. Instalación con Docker (alternativa).....	15

11. Preparación para producción	15
12. Operación, actualización y desinstalación.....	16
12.1 Comandos de uso frecuente.....	16
12.2 Actualizar	16
12.3 Desinstalar	17
13. Solución de problemas	17
14. Lista de verificación final	18

Este manual explica cómo instalar, configurar y verificar el Gateway LLM en una **máquina Linux**, con especial atención a las máquinas de prueba que ofrece GitHub. Está dirigido a quien instala o evalúa el gateway. Para el uso diario (enviar consultas, administrar claves, leer los registros) consulte el **Manual de usuario**.

1. Entornos de instalación

El gateway se instala igual en cualquier Linux. Este manual cubre tres entornos:

Opción	Entorno	Cuando usarla	Sección
A	GitHub Codespaces — máquina Linux en la nube, dentro de GitHub, con terminal en el navegador	Probar o demostrar el gateway sin instalar nada en un equipo propio	4
B	Máquina Linux propia — servidor, máquina virtual o equipo con Ubuntu o Debian	Instalación en un servidor de pruebas	5
C	GitHub Actions — máquina Linux temporal que GitHub crea en cada subida de código	Verificación automática, sin intervención	6

En los tres casos se usa el mismo instalador, `scripts/instalar.sh`, y la misma comprobación final, `scripts/prueba_humo.sh`.

2. Qué se instala

El gateway es un servicio web (FastAPI) que recibe todas las consultas a modelos de lenguaje por un único punto de entrada, `POST /v1/chat`, y aplica cuatro controles de seguridad del marco OWASP Top 10 for LLM Applications 2025.

Elemento	Descripción
<code>gateway/</code>	Código del servicio
<code>scripts/instalar.sh</code>	Instalador automático
<code>scripts/prueba_humo.sh</code>	Comprobación de que la instalación funciona
<code>venv/</code>	Entorno virtual con las dependencias (lo crea el instalador)
<code>.env</code>	Configuración (la crea el instalador; nunca se sube a GitHub)
<code>.gw_key</code>	Primera clave de cliente (la crea el instalador; nunca se sube a GitHub)
<code>logs/gateway.jsonl</code>	Registro de auditoría (se crea al arrancar)

Todo queda dentro de la carpeta del proyecto: no se instala nada en el sistema ni se necesitan permisos de administrador, salvo para los prerequisites de la sección 3.

La instalación no requiere API keys de ningún proveedor: usa un **proveedor simulado** que permite comprobar todo el funcionamiento sin costo.

3. Prerrequisitos

3.1 Software

Requisito	Versión	Para qué
Linux	Ubuntu 22.04 o 24.04, Debian 12, o equivalente	Sistema operativo
Python	3.10, 3.11, 3.12 o 3.13	Ejecutar el gateway
Módulo venv de Python	El de la versión instalada	Crear el entorno virtual
pip	Cualquiera reciente	Instalar dependencias
git	2.x	Descargar el código
curl	Cualquiera	Probar el servicio
make	Cualquiera (opcional)	Atajos de arranque y verificación

Instalación de todos los prerrequisitos en Ubuntu o Debian:

```
sudo apt-get update
sudo apt-get install -y python3 python3-venv python3-pip git curl make
```

Comprobación:

```
python3 --version
git --version
curl --version | head -n 1
```

Nota: en Ubuntu, el módulo venv viene en un paquete aparte (python3-venv). Si falta, el instalador se detiene y lo indica.

3.2 Recursos de la máquina

Recurso	Mínimo
Procesador	1 núcleo
Memoria	512 MB libres
Disco	200 MB (código, entorno virtual y registros)

Recurso	Mínimo
Puerto	8000 libre (se puede cambiar)

3.3 Red

Destino	Cuando se necesita
github.com	Descargar el código
pypi.org y files.pythonhosted.org	Instalar las dependencias
api.openai.com, api.anthropic.com, generativelanguage.googleapis.com	Solo si se conecta el proveedor correspondiente

Una vez instalado, el gateway con el proveedor simulado funciona sin conexión a internet.

3.4 Versiones verificadas

El instalador y las 63 pruebas se ejecutaron correctamente en Ubuntu 24.04 con Python 3.10, 3.11, 3.12 y 3.13. Las dependencias que se instalan son:

Paquete	Uso
fastapi, uvicorn	Servicio web
pydantic	Validación de solicitudes y protección de claves en memoria
slowapi	Límite de solicitudes por cliente
httpx	Llamadas a los proveedores
python-dotenv	Lectura del archivo .env
pytest, rich	Pruebas y reporte de evidencia

4. Opción A — GitHub Codespaces

Un codespace es una máquina Linux que GitHub crea a partir del repositorio y a la que se accede desde el navegador. El repositorio incluye el archivo `.devcontainer/devcontainer.json`, que le indica a GitHub que prepare la máquina con Python 3.12 y **ejecute el instalador automáticamente**.

Paso 1 · Crear el codespace

- Abra el repositorio en GitHub.
- Pulse el botón verde **Code**, pestaña **Codespaces**, y luego **Create codespace on main**.

- Espere uno o dos minutos. Se abre un editor en el navegador con una terminal en la parte inferior.

Durante la creación, GitHub ejecuta `bash scripts/instalar.sh` (el detalle se describe en la sección 5, paso 3). Compruebe que terminó:

```
ls -d venv .env .gw_key
```

Si falta alguno de los tres, ejecute el instalador a mano. No hay riesgo en repetirlo:

```
bash scripts/instalar.sh
```

Paso 2 · Verificar la instalación

```
bash scripts/prueba_humo.sh
```

El resultado esperado está en la sección 7.

Paso 3 · Arrancar el gateway

```
make protegido
```

Equivalente sin `make`: `venv/bin/uvicorn gateway.main:app --port 8000`.

GitHub detecta el puerto 8000 y muestra un aviso para abrirlo. Deje esta terminal abierta mientras use el gateway.

Paso 4 · Probar desde una segunda terminal

Abra otra terminal con el botón + del panel inferior:

```
export GW_KEY=$(cat .gw_key)
curl -s -X POST http://localhost:8000/v1/chat \
  -H "Authorization: Bearer $GW_KEY" -H "Content-Type: application/json" \
  -d '{"mensaje":"¿Puedo devolver un producto que compré hace 20 días?}"'
```

Paso 5 · Acceder desde el navegador o desde fuera

En la pestaña **PORTS** del panel inferior aparece el puerto 8000 con su dirección, de la forma

`https://NOMBRE-DEL-CODESPACE-8000.app.github.dev`.

Qué quiere hacer	Cómo
Abrir la documentación interactiva	Abra la dirección del puerto y agregue <code>/docs</code>
Comprobar que responde	Abra la dirección del puerto y agregue <code>/health</code>
Permitir que otra persona acceda	Clic derecho en el puerto, Port Visibility , Public

Por defecto el puerto es **privado**: solo usted, con su sesión de GitHub iniciada, puede abrirlo. Aunque lo haga público, el gateway sigue exigiendo la clave **gw_** en cada consulta.

Paso 6 · Claves de proveedores como secretos (opcional)

Para usar un modelo real, no escriba la clave en un archivo. Guárdela como secreto de Codespaces:

- En GitHub, foto de perfil, **Settings, Codespaces, Codespaces secrets, New secret**.
- Nombre: **OPENAI_API_KEY** (o **ANTHROPIC_API_KEY**, **GOOGLE_API_KEY**). Valor: la clave. Seleccione este repositorio.
- Agregue otro secreto **PROVEEDOR_PRINCIPAL** con el valor **openai**, o edite esa línea en **.env**.
- Reinicie el codespace: los secretos nuevos solo están disponibles al crearlo o reiniciarlo.

Los secretos llegan al gateway como variables de entorno, que tienen prioridad sobre el archivo **.env**.

Paso 7 · Detener y eliminar

- **Detener el gateway:** **Ctrl + C** en su terminal.
- **Detener el codespace:** se detiene solo tras 30 minutos sin uso. Para detenerlo antes, en github.com/codespaces use el menú ... y **Stop codespace**.
- **Eliminarlo:** en esa misma página, **Delete**. Se pierden **.env** y **.gw_key**; al crear uno nuevo, el instalador emite otra clave.

***Nota:** las cuentas personales de GitHub incluyen una cuota mensual gratuita de Codespaces (a la fecha de este manual, 120 horas de cómputo y 15 GB de almacenamiento en el plan Free). Detenga o elimine el codespace cuando no lo use.*

5. Opción B — Máquina Linux propia

Paso 1 · Instalar los prerequisites

```
sudo apt-get update
sudo apt-get install -y python3 python3-venv python3-pip git curl make
```

Paso 2 · Descargar el código

```
git clone https://github.com/cesarmv85-del/fundamentos-arquitectura-llm-gateway-owasp.git
cd fundamentos-arquitectura-llm-gateway-owasp
```

Sin **git**: en la página del repositorio, **Code → Download ZIP**, descomprima y entre en la carpeta. El instalador funciona igual.

Paso 3 · Ejecutar el instalador

```
bash scripts/instalar.sh
```

Salida real de una instalación limpia (tarda menos de un minuto):

```
→ Comprobando prerequisites
✓ Python 3.12.3 · git 2.43.0 · curl disponible
→ Creando entorno virtual en venv/
→ Instalando dependencias (requirements.txt)
✓ Dependencias instaladas en venv/
✓ Archivo .env creado a partir de .env.example
✓ Clave emitida para 'equipo_demo': hash en .env, clave en .gw_key (permisos 600, ignorado por git)
→ Ejecutando las pruebas
63 passed, 1 warning in 0.75s
✓ Sin secretos detectados en archivos + historial git + logs/

Instalación completa. Sigüientes pasos:
```

Qué hace el instalador, en orden:

- **Comprueba los prerequisites** y se detiene con un mensaje claro si falta alguno.
- **Crea el entorno virtual** `venv/` e instala las dependencias.
- **Crea `.env`** a partir de `.env.example`, con permisos solo para el propietario.
- **Emita la primera clave de cliente:** escribe su huella (hash) en `.env` y guarda la clave en `.gw_key`.
- **Ejecuta las 63 pruebas y el escáner de secretos.**

Opciones del instalador:

Opción	Efecto
<code>bash scripts/instalar.sh --sin-pruebas</code>	Omite las pruebas
<code>CLIENTE=equipo_ventas bash scripts/instalar.sh</code>	Nombre del primer cliente (por defecto <code>equipo_demo</code>)
<code>PYTHON=python3.11 bash scripts/instalar.sh</code>	Usa un intérprete concreto

El instalador se puede **ejecutar varias veces sin riesgo**: conserva el `.env` existente y no emite una clave nueva si ya hay clientes configurados.

Paso 4 · Verificar

```
bash scripts/prueba_humo.sh
```

Paso 5 · Arrancar el gateway

Hay tres formas, según el uso:

Forma	Comando	Cuando
En primer plano	<code>make protegido</code>	Pruebas y demostraciones; se detiene con <code>Ctrl + C</code>
En segundo plano	<code>make iniciar</code>	Dejarlo funcionando al cerrar la terminal
Accesible desde otras máquinas	<code>make iniciar</code> <code>HOST=0.0.0.0</code>	Servidor de pruebas compartido

Control del gateway en segundo plano:

```
make iniciar    # arranca; la salida queda en logs/servidor.out
make estado    # {"status":"ok"} si está en funcionamiento
make detener   # lo detiene
```

Para usar otro puerto: `make iniciar PUERTO=8080`.

Importante: con `HOST=0.0.0.0` el gateway acepta conexiones de la red. Abra el puerto solo a las máquinas necesarias y recuerde que el gateway no cifra por sí mismo: en una red no confiable, publíquelo detrás de HTTPS (sección 11).

Si el servidor tiene cortafuegos `ufw` y quiere permitir el acceso desde la red interna:

```
sudo ufw allow from 10.0.0.0/8 to any port 8000 proto tcp
```

Instalación manual (alternativa al instalador)

Si prefiere hacer cada paso a mano:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python scripts/generar_clave_cliente.py equipo_demo
```

El último comando muestra la clave `gw_...` y una línea `equipo_demo:hash`. Copie esa línea en `.env`, a la derecha de `GATEWAY_CLIENT_KEYS_SHA256=`, y guarde la clave en un lugar seguro. Después arranque con `uvicorn gateway.main:app --port 8000`.

6. Opción C — GitHub Actions

El repositorio incluye el flujo `.github/workflows/ci.yml`. En cada subida de código, GitHub crea una máquina Linux (`ubuntu-latest`), instala el gateway y lo prueba, sin intervención.

Paso del flujo	Qué comprueba
<code>bash scripts/instalar.sh</code>	El instalador funciona en una máquina limpia; pasan las 63 pruebas y el escáner de secretos, incluido el historial de git
<code>bash scripts/prueba_humo.sh</code>	El gateway arranca y responde a solicitudes reales
<code>scripts/demo_antes_despues.py</code>	Cada mitigación se comporta distinto con y sin protección

Cómo usarlo:

- **Ver el resultado:** pestaña **Actions** del repositorio. Un check verde indica que todo pasó.
- **Lanzarlo a mano:** en **Actions**, seleccione **Pruebas y secretos, Run workflow**.
- **Ver el detalle:** abra una ejecución y despliegue cada paso para ver su salida.

La máquina se destruye al terminar: sirve para verificar, no para dejar el gateway en funcionamiento.

7. Verificación de la instalación

La comprobación es la misma en los tres entornos:

```
bash scripts/prueba_humo.sh
```

El script arranca el gateway si no está en marcha, hace seis comprobaciones con solicitudes reales y lo detiene. Resultado esperado:

```
✓ El servicio responde (GET /health)           HTTP 200
✓ Consulta legítima con clave (POST /v1/chat)   HTTP 200
✓ Consulta sin clave → rechazada                 HTTP 401
✓ Prompt injection → bloqueada (LLM01)          HTTP 400
✓ max_tokens desmedido → rechazado (LLM10)       HTTP 422
✓ Las cuatro mitigaciones están activas
```

Instalación verificada: el gateway funciona correctamente.

Si alguna línea aparece con **X**, el script termina con error e indica el código obtenido y el esperado.

Comprobaciones adicionales:

Comando	Qué verifica	Resultado esperado
<code>make test</code>	Las 63 pruebas, una por una	63 passed (puede añadir 1 warning, inofensivo)
<code>make secretos</code>	Que no hay claves en el código, el historial ni los registros	✓ Sin secretos detectados...

Comando	Qué verifica	Resultado esperado
make evidencia	Comparación antes/después de cada mitigación	Tabla con seis escenarios en ✓

Consulta manual, con el gateway en marcha:

```
export GW_KEY=$(cat .gw_key)
curl -s http://localhost:8000/health
curl -s -X POST http://localhost:8000/v1/chat \
  -H "Authorization: Bearer $GW_KEY" -H "Content-Type: application/json" \
  -d '{"mensaje":"¿Puedo devolver un producto que compré hace 20 días?}"'
```

```
{"status":"ok"}
{"request_id":"df8dd6c0d7934cc7","respuesta":"Puedes devolver tu producto dentro de 30 días desde la compra, con boleta y sin uso.", "proveedor":"simulado", "modelo":"modelo-ingenuo-v1", "simulado":true, "degradado":false, "tokens_entrada":148, "tokens_salida":21, "latencia_ms":0}
```

8. Configuración

La configuración está en el archivo **.env**. Edítelo con **nano .env** y **reinicie el gateway** después de cada cambio. Una variable de entorno del sistema con el mismo nombre tiene prioridad sobre el archivo.

8.1 Proveedores

Variable	Valor por defecto	Descripción
PROVEEDOR_PRINCIPAL	simulado	simulado, openai, anthropic, google u ollama
PROVEEDOR_RESPALDO	(vacío)	Proveedor al que se recurre si el principal falla
OPENAI_API_KEY · ANTHROPIC_API_KEY · GOOGLE_API_KEY	(vacío)	Clave del proveedor correspondiente
MODELO_OPENAI	gpt-4o-mini	Modelo a usar con OpenAI
MODELO_ANTHROPIC	claude-haiku-4-5	Modelo a usar con Anthropic
MODELO_GOOGLE	gemini-2.5-flash	Modelo a usar con Google
MODELO_OLLAMA	llama3.2	Modelo local de Ollama
OLLAMA_BASE_URL	http://localhost:11434	Dirección del servidor Ollama

Variable	Valor por defecto	Descripción
TIMEOUT_UPSTREAM_SEG	20	Segundos de espera máxima al proveedor

8.2 Clientes y límites de consumo

Variable	Valor por defecto	Descripción
GATEWAY_CLIENT_KEYS_SHA256	(la escribe el instalador)	Clientes autorizados: nombre:hash, separados por coma
RATE_LIMIT	5/minute	Solicitudes permitidas por clave. Otros ejemplos: 100/hour, 20/minute
RATE_LIMIT_POR	clave	Deje clave. El valor ip existe solo para demostrar por qué no conviene
RATE_LIMIT_STORAGE_URI	memory://	Dónde se guarda el contador. Con varias réplicas use redis://servidor:6379
MAX_TOKENS_TECHO	512	Máximo de tokens de salida que puede pedir una solicitud
MAX_CARACTERES_MENSAJE	4000	Largo máximo del mensaje
MAX_BYTES_BODY	16384	Tamaño máximo de la solicitud completa

8.3 Registros, navegador y entorno

Variable	Valor por defecto	Descripción
ENTORNO	desarrollo	Con produccion se activan las exigencias de la sección 11
LOG_ARCHIVO	logs/gateway.jsonl	Archivo del registro de auditoría
LOG_CONSOLA	true	Mostrar también los registros en la terminal
CORS_ORIGINS	http://localhost:5173	Sitios web autorizados a llamar al gateway desde un navegador

8.4 Interruptores de demostración

Variable	Valor por defecto	Descripción
MITIGACION_LLM01 · LLM02 · LLM07 · LLM10	true	Apagar una mitigación para mostrar el ataque sin protección
SIMULAR_FALLA_UPSTREAM	(vacío)	timeout, error_500 o error_con_clave: simula una caída del proveedor

No hace falta editar `.env` para la demostración: `make linea-base M=LLM01` arranca el gateway con esa mitigación apagada solo para esa ejecución.

Importante: con `ENTORNO=produccion` el gateway se niega a arrancar si algún interruptor está apagado.

9. Conectar un proveedor real

Edite `.env`, indique el proveedor y su clave, y reinicie.

OpenAI:

```
PROVEEDOR_PRINCIPAL=openai
OPENAI_API_KEY=su_clave_de_openai
```

Anthropic:

```
PROVEEDOR_PRINCIPAL=anthropic
ANTHROPIC_API_KEY=su_clave_de_anthropic
```

Google Gemini:

```
PROVEEDOR_PRINCIPAL=google
GOOGLE_API_KEY=su_clave_de_google
```

Ollama (modelo local, sin clave ni costo):

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull llama3.2
```

```
PROVEEDOR_PRINCIPAL=ollama
```

Recomendaciones:

- **Configure un respaldo.** Por ejemplo `PROVEEDOR_RESPALDO=ollama`: si el principal falla, el gateway responde con el respaldo y marca la respuesta con `degradado: true`.

- **No suba la clave a GitHub.** El archivo `.env` está excluido por `.gitignore`. En Codespaces use secretos (sección 4, paso 6).
- **En servidores, prefiera un archivo de secreto.** En lugar de la variable, indique la ruta:
`OPENAI_API_KEY_FILE=/run/secrets/openai_api_key`. Así la clave no queda en el entorno del proceso.

Si la clave falta o es inválida, el gateway **no** responde con datos simulados: devuelve un error **502** con un mensaje claro, para que el fallo no pase desapercibido.

***Nota:** la conexión con proveedores reales no se probó con claves reales al preparar este manual; sí se verificó el comportamiento ante clave ausente (502), proveedor caído (503) y uso del respaldo.*

10. Instalación con Docker (alternativa)

Requiere Docker y un archivo `.env` con al menos un cliente. Si no lo tiene, ejecute antes `bash scripts/instalar.sh --sin-pruebas`.

```
docker build -t gateway-llm .
docker run --rm -p 8080:8080 --env-file .env gateway-llm
```

El gateway queda en `http://localhost:8080`. La imagen arranca en modo producción, se ejecuta con un usuario sin privilegios y no contiene claves: el archivo `.env` se entrega al arrancar.

***Nota:** el modo producción y el comando de arranque del contenedor se verificaron ejecutándolos directamente. La construcción de la imagen con `docker build` no pudo ejecutarse en el entorno donde se preparó este manual; pruébela antes de un uso real.*

11. Preparación para producción

Con `ENTORNO=produccion` el gateway aplica estas reglas al arrancar:

Regla	Si no se cumple
Las cuatro mitigaciones deben estar activas	No arranca: Mitigaciones desactivadas en producción: [...]
<code>SIMULAR_FALLA_UPSTREAM</code> debe estar vacío	No arranca
Debe existir al menos una clave de cliente	No arranca: No hay claves de cliente configuradas
La documentación /docs se desactiva	Responde 404

Antes de exponer el servicio:

- **Use un proveedor real**, no el simulado.
- **Publique el gateway detrás de HTTPS** (balanceador o proxy inverso como Nginx o Caddy).

- Emita una clave por aplicación y elimine `.gw_key` del servidor una vez entregada la clave.
- Con más de una réplica, use Redis para el contador de cuotas:

```
venv/bin/pip install redis
```

```
RATE_LIMIT_STORAGE_URI=redis://servidor:6379
```

Sin Redis, cada réplica llevaría su propia cuenta. Con Redis el límite se comparte: en una prueba con dos réplicas y límite de 5 por minuto, pasaron exactamente 5 solicitudes entre ambas.

- Guarde las claves de los proveedores en un gestor de secretos y móntelas como archivo (`*_FILE`).
- Envíe `logs/gateway.jsonl` a un sistema central de registros y defina alertas sobre los resultados `bloqueado_llm01`, `fuga_bloqueada_llm07` y `rate_limit_excedido`.
- Limite `CORS_ORIGINS` a los sitios reales que consumirán el gateway.

El diagrama `docs/arquitectura/06_despliegue_produccion.png` muestra una propuesta completa sobre Google Cloud.

12. Operación, actualización y desinstalación

12.1 Comandos de uso frecuente

Comando	Efecto
<code>make instalar</code>	Ejecuta el instalador
<code>make verificar</code>	Prueba de humo
<code>make protegido</code>	Arranca en primer plano con todas las mitigaciones
<code>make iniciar</code> · <code>make estado</code> · <code>make detener</code>	Gateway en segundo plano
<code>make linea-base M=LLM07</code>	Arranca con una mitigación apagada (demostración)
<code>make falla-upstream F=timeout</code>	Arranca simulando una caída del proveedor
<code>make test</code> · <code>make secretos</code> · <code>make evidencia</code>	Pruebas, escáner y reporte antes/después

Todos aceptan `PUERTO=` y `HOST=`. Sin `make`, los comandos equivalentes están dentro del archivo `Makefile`.

12.2 Actualizar

```
make detener
git pull
bash scripts/instalar.sh
```



```
make iniciar
```

El instalador actualiza las dependencias y conserva `.env`, `.gw_key` y los registros.

12.3 Desinstalar

```
make detener
cd ..
rm -rf fundamentos-arquitectura-llm-gateway-owasp
```

No queda nada fuera de esa carpeta. Las claves `gw_` emitidas dejan de funcionar.

13. Solución de problemas

Síntoma	Causa probable	Solución
X Se requiere Python 3.10 o superior	Python ausente o antiguo	<code>sudo apt-get install -y python3 python3-venv python3-pip</code>
X Falta el módulo <code>venv</code> de Python	Falta el paquete <code>python3-venv</code>	<code>sudo apt-get install -y python3-venv</code>
X Falta <code>'git'</code> o <code>'curl'</code>	Herramienta no instalada	<code>sudo apt-get install -y git curl</code>
<code>make: command not found</code>	<code>make</code> no instalado	<code>sudo apt-get install -y make</code> , o use los comandos equivalentes
Error al descargar dependencias	Sin salida a <code>pypi.org</code>	Revise la red o el proxy (<code>HTTPS_PROXY</code>)
<code>address already in use</code> al arrancar	El puerto está ocupado	<code>make protegido PUERTO=8001</code> , o <code>make detener</code> si quedó uno en segundo plano
<code>uvicorn: command not found</code>	Entorno virtual no activo	Use <code>make protegido</code> o <code>venv/bin/uvicorn</code> ...
<code>Permission denied</code> al ejecutar un script	Falta el permiso de ejecución	<code>bash scripts/instalar.sh</code> , o <code>chmod +x scripts/*.sh</code>
Todas las consultas devuelven 401	La clave no coincide con el hash de <code>.env</code>	Compruebe <code>echo \$GW_KEY</code> y que <code>.env</code> tenga la línea <code>GATEWAY_CLIENT_KEYS_SHA256</code>
401 después de editar <code>.env</code>	No se reinició el gateway	<code>make detener && make iniciar</code> , o <code>Ctrl + C</code> y arrancar de nuevo
429 al poco de empezar	Se superó el límite de 5 solicitudes por minuto	Espere un minuto o aumente <code>RATE_LIMIT</code>

Síntoma	Causa probable	Solución
502 con un proveedor real	Falta la API key o es inválida	Revise la clave del proveedor
503 con Ollama	Ollama no está en ejecución o falta el modelo	<code>ollama serve</code> y <code>ollama pull llama3.2</code>
504	El proveedor tardó más que <code>TIMEOUT_UPSTREAM_SEG</code>	Aumente el valor o configure un respaldo
'redis' prerequisite not available	Se configuró <code>redis://</code> sin instalar el paquete	<code>venv/bin/pip install redis</code>
No arranca con <code>ENTORNO=produccion</code>	Falta un cliente o hay un interruptor apagado	Lea el mensaje <code>RuntimeError</code> y corrija <code>.env</code>
No puedo conectarme desde otra máquina	El gateway escucha solo en <code>127.0.0.1</code>	<code>make iniciar</code> <code>HOST=0.0.0.0</code> y revise el cortafuegos
En Codespaces el puerto pide iniciar sesión	El puerto es privado	Inicie sesión en GitHub o cambie la visibilidad en PORTS
En Codespaces no aparece <code>.gw_key</code>	El instalador no terminó	<code>bash scripts/instalar.sh</code>
Un secreto de Codespaces no llega al gateway	El codespace no se reinició	Deténgalo y vuelva a iniciarlo

Para investigar una solicitud concreta, busque su `request_id` en el registro:

```
grep 'df8dd6c0d7934cc7' logs/gateway.jsonl
```

La salida del gateway en segundo plano está en `logs/servidor.out`.

14. Lista de verificación final

- ✓ `python3 --version` muestra 3.10 o superior
- ✓ `bash scripts/instalar.sh` termina con **Instalación completa**
- ✓ Existen `venv/`, `.env` y `.gw_key`
- ✓ `bash scripts/prueba_humo.sh` muestra seis líneas con ✓
- ✓ Al arrancar, la primera línea muestra las cuatro mitigaciones en **true**
- ✓ `curl -s http://localhost:8000/health` responde `{"status":"ok"}`
- ✓ Una consulta con la clave devuelve **200**; sin la clave, **401**
- ✓ `git status` no muestra `.env` ni `.gw_key` como archivos por subir
- ✓ En GitHub, la pestaña **Actions** muestra el check verde